

Spring Algorithm (春季算法)

Formal name: Spring Self-Evolving Gatekeeper (SSEG) **Ab-**

breviation: Spring **Location:** /g/Xiaogan_Supercomputing_data/SCX/theory/self_evolu

Paper target: Nature Computational Science (Paper 2 of two-paper strategy)

Why “Spring”

Spring is the season of **resurrection** (复苏).

Nature's Spring → SCX Spring Algorithm

Winter dormancy	→ Structures below threshold <code>_keep</code> — dormant, not dead
Spring thaw	→ Gatekeeper <code>S_t</code> updates (new knowledge arrives)
Re-blooming	→ Old structures re-scored — those crossing <code>_keep</code> are RES
Each spring cycle	→ Each iteration: <code>S_t</code> re-evaluates <code>M_t</code>
Perennial growth	→ Memory bank <code>M_t</code> grows monotonically, never pruned
Deeper roots	→ DFT/experimental ground truth (physical anchor prevents c

Tagline: *“Winter does not kill — it waits for spring. Every discarded structure carries the seed of its own resurrection.”*

Algorithm Sketch

Spring(`S_0`, `M_0`, `D_stream`):

```

for t = 0, 1, 2, ...:
    // Judge: score incoming data
    for (x, y) in D_t:
        score = S_t(x, y)                // gatekeeper evaluates
        M_t.insert(x, y, score, metadata) // store always, never delete
        if score >= _keep:
            train_NEP(x, y)                // only high-score → training

    // Update: NEP student learns, gatekeeper evolves
    _{t+1} = NEP_update(_t, M_t)          // student learns from selected data
    S_{t+1} = Gatekeeper_update(S_t, M_t, f_{_{t+1}}) // gatekeeper sees through

    // Resurrect: re-score old dormant structures
    for (x, y, old_score) in M_t where old_score < _keep:
        new_score = S_{t+1}(x, y)
        if new_score >= _keep:
            RESURRECT(x, y)                // spring brings it back to life
            train_NEP(x, y)

```

Core Theorems

Theorem	Statement	Status
Spring-1 (Convergence)	Under C1-C7, $(S_t, _t) \rightarrow (S, _)$ a.s. — the gatekeeper converges to a self-consistent fixed point	Formalized in 06_fixed_point_convergence.md

Theorem	Statement	Status
Spring-2 (Completeness)	Under physical constraints (finite data, finite precision, finite compute), finite T^* such that system reaches -approximate fixed point	Formalized in <code>07_completeness.md</code>

Two-Paper Strategy

	Paper 1: SCX	Paper 2: Spring
Title	<i>State-Conditioned Expertise: A Complete Theory of Label Noise Detection</i>	<i>Spring: A Self-Evolving Gatekeeper with Provable Convergence</i>
Core Target	Thm 1, 2, 3 JMLR / TMLR	Spring-1, Spring-2 Nature Computational Science
Story	“Noise detection needs state-conditioning — we prove necessity + sufficiency”	“An AI that learns what ‘good data’ means — and brings discarded data back to life”